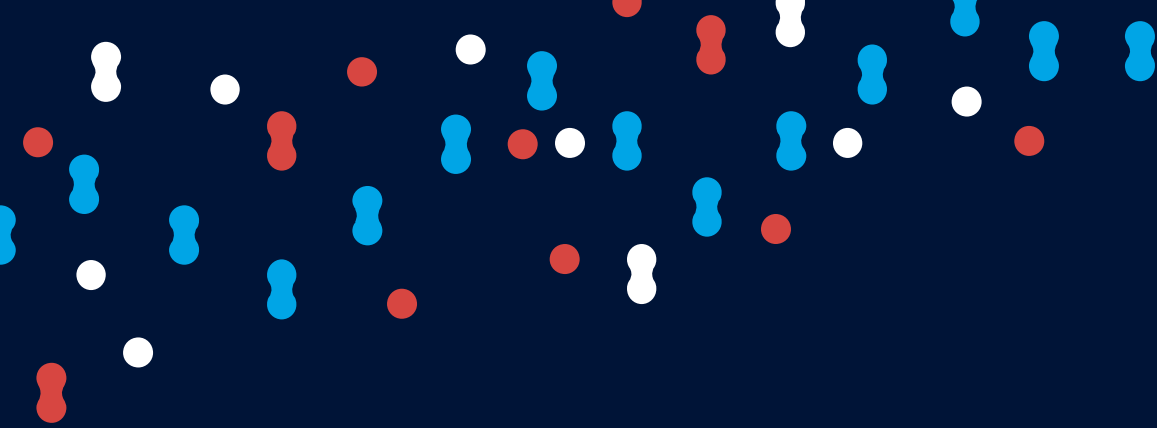




Autotasker: Self-Service Task Scheduler

Team Members Introduction



Kirthick Easwar B
Technical Analyst
LDP26



Sharath Chinnarigari
Software Developer
LDP26



Vansh Ajwani
Software Developer
LDP26



Thanigaivel Prakash
Software Developer
LDP26



Swaminath C
Technical Analyst
LDP26

Problem statement



Problem statement



Problem Statement

Business users requires ION Support team for the entire **task lifecycle**. Meanwhile, ION Support team is heavily consumed **by repetitive, low complexity, attention-intensive requests**, leading to inefficient use of resources **and long change to production time** for business users

Breakdown for last year (2025) in Spectrum: Time spent by Support team on resolving these tickets

Task Category	Count	Hours per Task	Total Hours
New Batch Reports - Required Scheduling	41	4	164
Updates / Autotask updates / Monitoring tasks	26	1.5	39
Total	67	-	203

Note:

- This does not include the waiting time for the client
- This does not include clients under the implementation project, which would be 2-3 times of the above effort

Existing Workflow

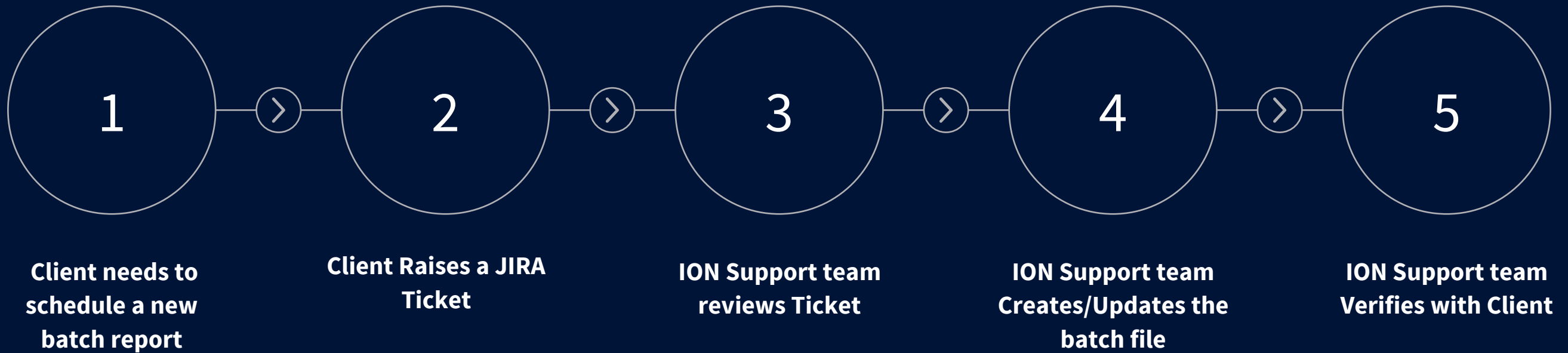


Existing Workflow

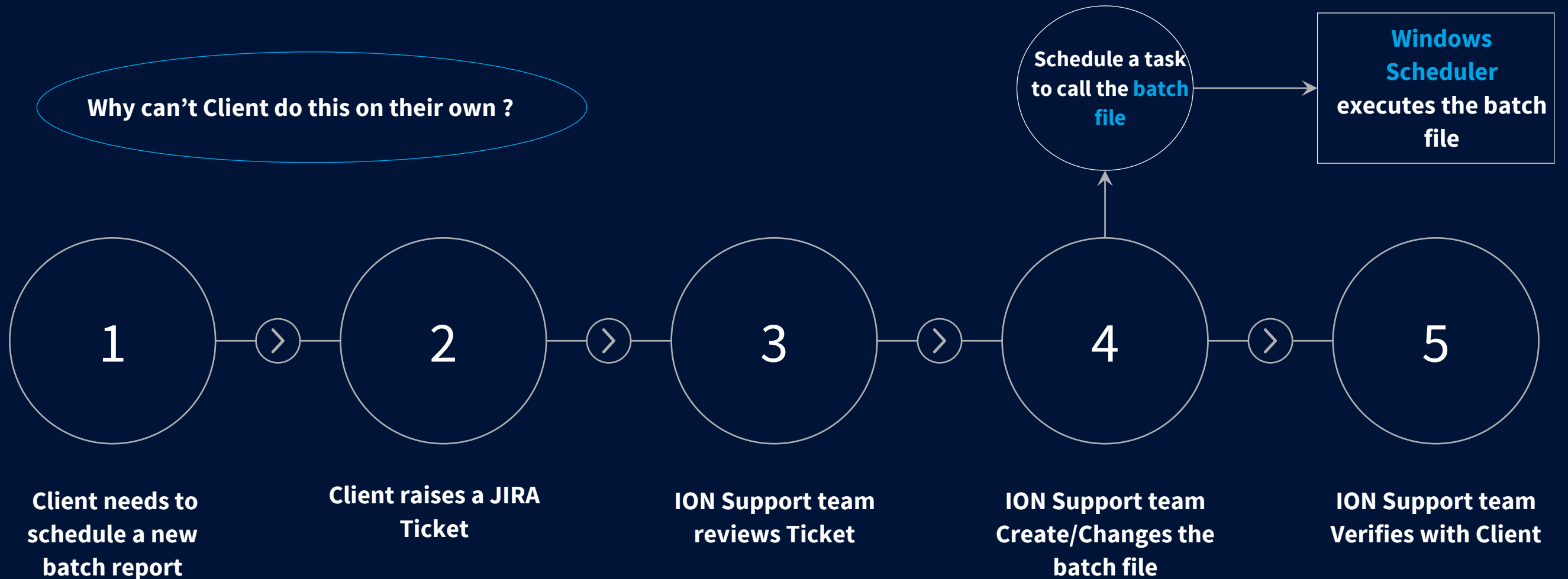
Schedule a new batch report



Existing Workflow



Schedule a new batch report



Schedule a new batch report



**Client needs to
schedule a new
batch report**

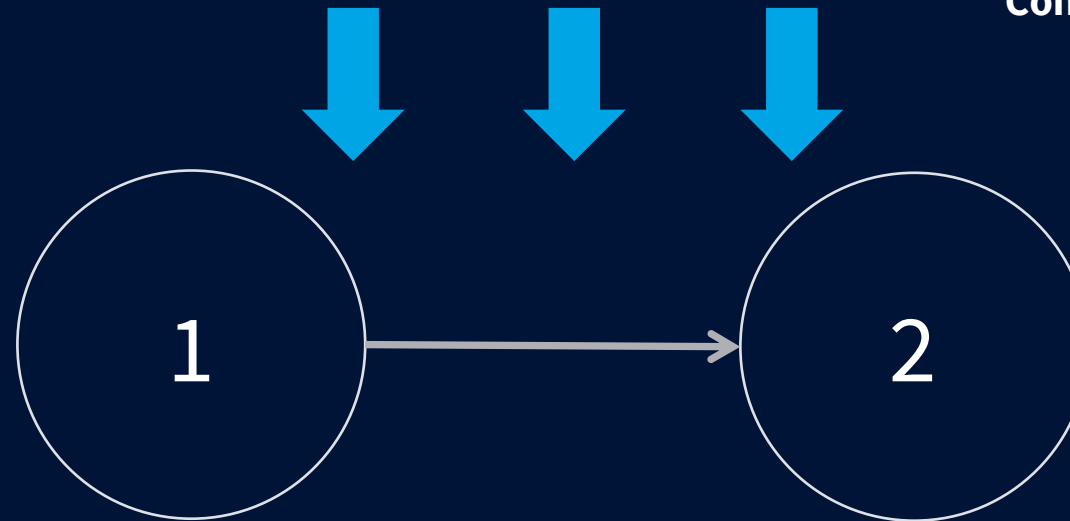
**Client raises a JIRA
ticket**

**ION Support team
reviews Ticket**

**ION Support team
Changes/update
Configurations**

**ION Support team
Verifies with Client**

Proposed Workflow:



**Client needs schedule a new
batch report**

**Client schedules
task**

Existing workflow



Update a task

Process: Submit IT ticket → Wait 1–2 days → IT updates task → Business verifies

Change to Production time : 1–2 days.

Pause a Task Temporarily

Process: Email IT → Wait for response → IT pauses task → Confirms with business

Change to Production time : 1–2 days



Troubleshoot a Failed Task

Process: User notices failure → Contacts IT → Waits for log access → Reviews with IT

Change to Production time : 4–8 hours



End-user profiles

Viewer

- **Read-only access** with very low risk.
- **Can only view** dashboards, tasks, schedules, reports and logs
- Cannot edit, upload, delete, or create any content.

Editor

- **Can create and update** content like tasks and schedules, view logs, and troubleshoot issues, **but cannot change system settings**, structures, or **manage others' access**.
- Restricted from deleting critical components.

Admin

- **Manage users and their Access controls:** Can add, remove and update the roles of the user
- Can edit critical structures like task type, etc
- Can create, update, monitor all tasks and schedules of all users and can view audit logs, troubleshoot issues

Approach



Approach

Tech stack and Architecture Diagram

Frontend

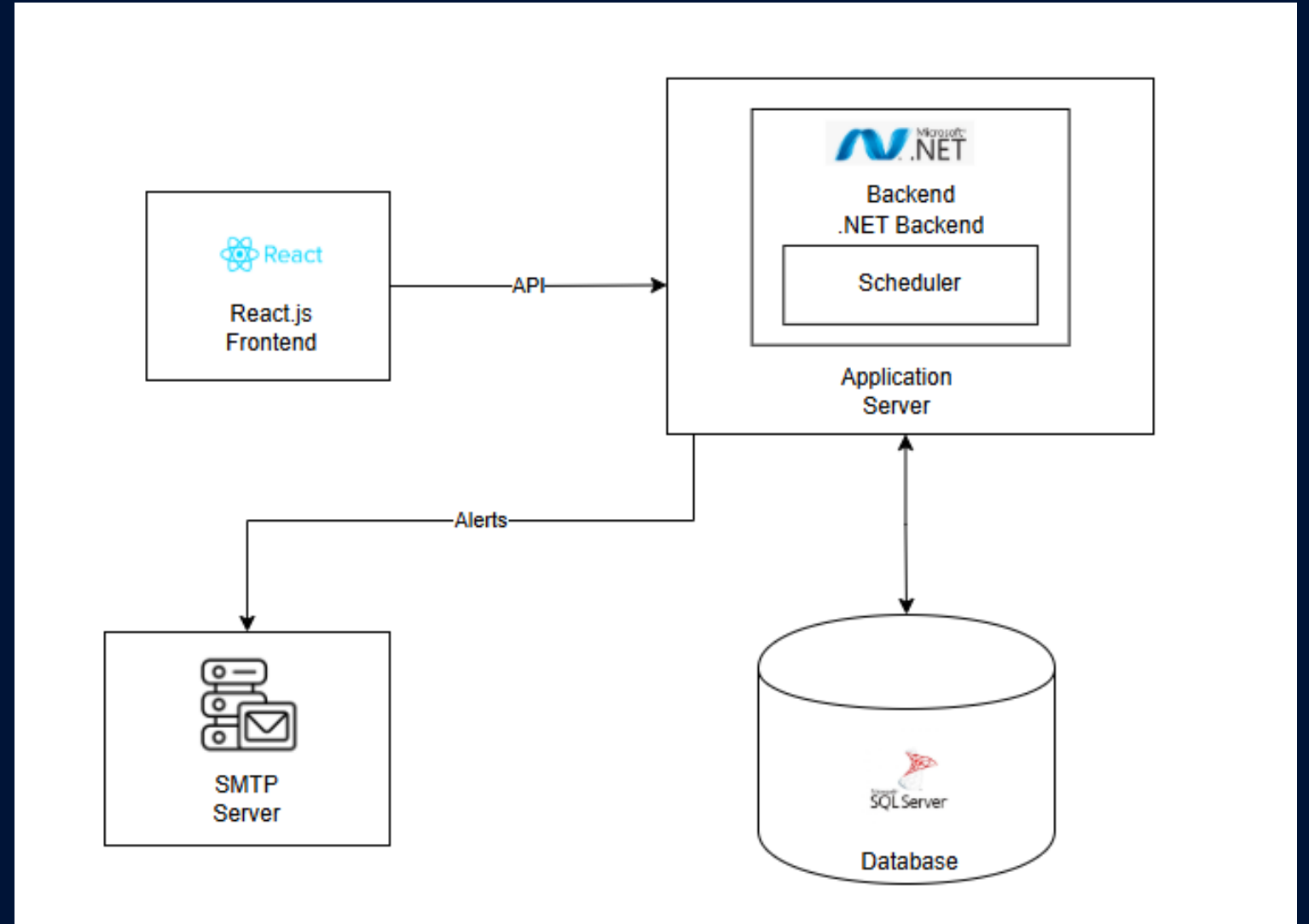
- React.js + Vite

Database

- Microsoft SQL

Backend

- Microsoft .Net (C#)





Task Scheduler – Behaviours and Edge Cases

Concurrency

- Same task: **No parallel runs**, next one is **queued**
- Different tasks: Up to **10 run simultaneously**

Misfire Handling Options

- **Run missed once**, then continue normally
- **Skip the missed execution**
- **Run all missed** executions on restart

Restart Behavior

- Publish app as a **Windows service**
- Set service to **Automatic startup** via PowerShell

Long-Running Jobs

- If schedule = every 2 min but job takes 3–4 min: Quartz **won't queue overlaps**
- Next run fires at the **next valid future time**



Tools used and tests performed

- **Tool used:** xUnit
- **Total no. of Unit tests done:** 62
- **Total no. of Integration tests done:** 10

Controller	Line Coverage	Branch Coverage	Controller
AuditsController	5 / 5 (100%)	— (no branches)	AuditsController
AuthController	34 / 34 (100%)	12 / 12 (100%)	AuthController
SchedulesController	33 / 34 (97%)	5 / 6 (83.3%)	SchedulesController
TaskHistoryController	5 / 5 (100%)	— (no branches)	TaskHistoryController
TasksController	48 / 50 (96%)	7 / 10 (70%)	TasksController
TaskTypesController	38 / 38 (100%)	6 / 6 (100%)	TaskTypesController
UsersController	32 / 32 (100%)	4 / 4 (100%)	UsersController

Benefits and KPIs

Benefits



Improved Time Efficiency



Greater Transparency in Operations



Lower Dependency on Support Teams



Stronger Process Governance



KPIs



Median Task Configuration Time



Change in Support Ticket Volume



Task Failure Rate

Future Scope



Configurable Batch Execution



System User Identity Selection



Integration with Workflow Tools



Configurable Export and Import Capabilities



AI-Assisted Troubleshooting

Future Scope

Challenges faced by us



New Tech Stack Ramp-Up



Unclear Initial Requirements



Frequent Changes Due to Edge Cases



Manual Testing and Testing Delays

**Challenges
faced by us**

Features Table and Access Control

Access / Module	Viewer	Editor	Admin
View Dashboard (Admin → admin, User → viewer/editor)	✓	✓	✓
View Tasks (all visible tasks)	✓	✓	✓
View Task Details	✓	✓	✓
View Audit	✓	✓	✓
View Troubleshoot (error tasks)	✓	✓	✓
Run Task (manual retry/run now)	X	✓	✓
Create Task	X	✓	✓
Edit Task	X	✓	✓
Delete Task	X	✓	✓
Create / Edit / Delete Schedule	X	✓	✓
Pause / Resume Schedule	X	✓	✓
View Task History	✓	✓	✓
Manage Task Types	X	X	✓
Manage Users (create/delete/assign roles)	X	X	✓
Change Own Password	✓	✓	✓



Live Demo



Q & A



Appendix



Our Approach

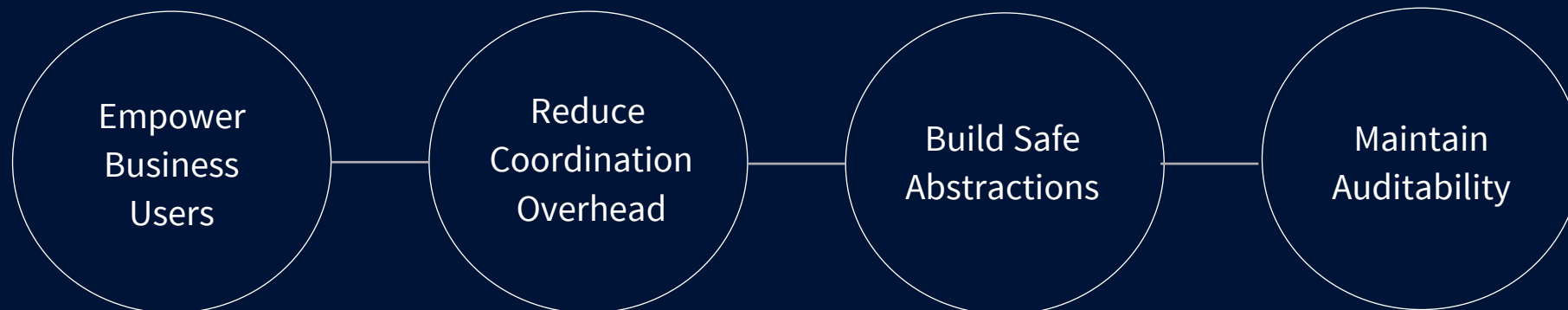
Scientific Thinking:

Hypothesis: If business users can schedule, update and monitors tasks via a self-service UI with RBAC + audit logs, then the average “change-to-production” time for scheduling actions will drop from 1–3 days to < 2 minutes for standard changes, without increasing failure rates

Measured by:

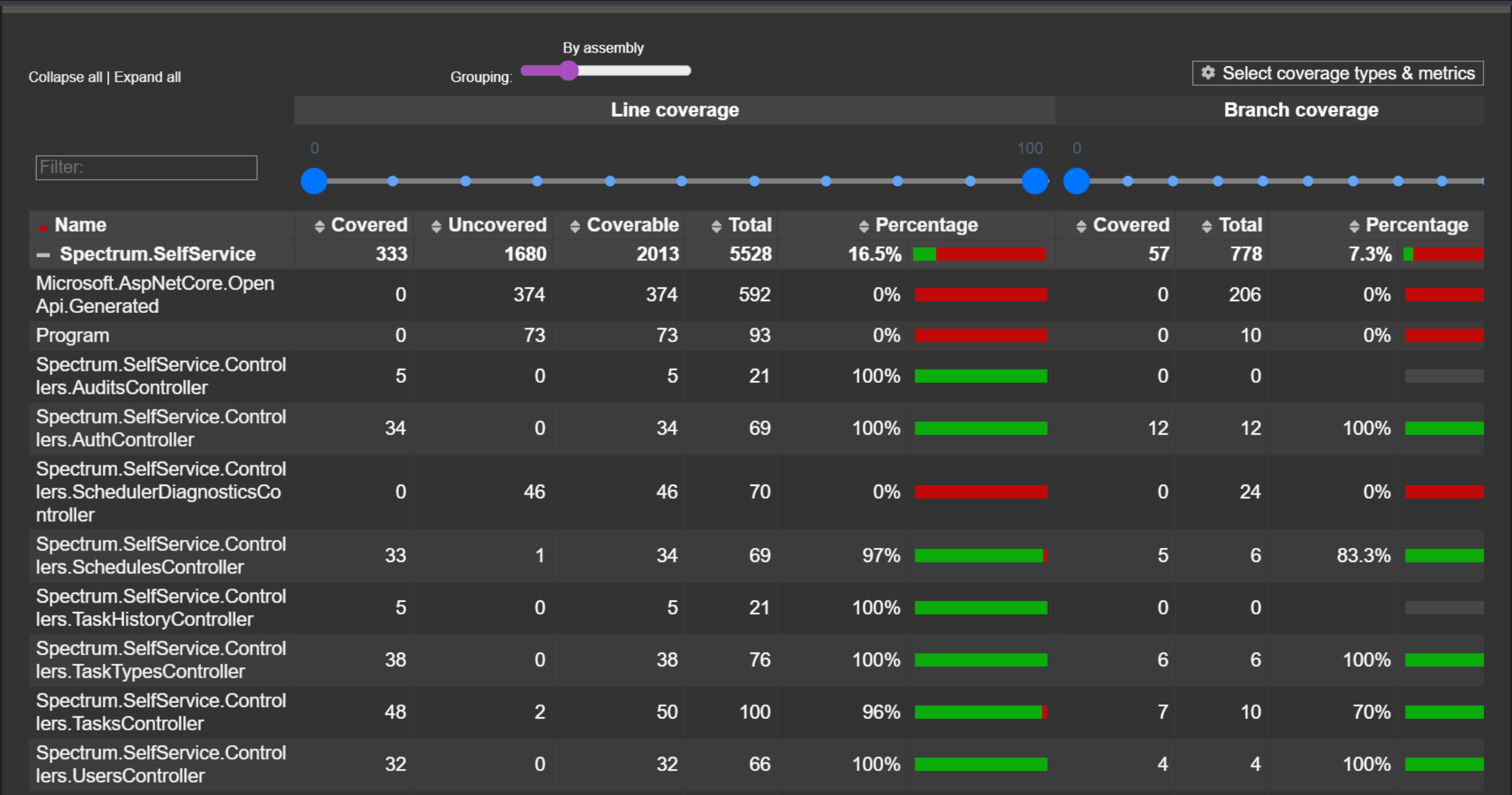
- Median time to schedule/update a task
- % changes in the number of tickets raised
- Task failure rate

First Principles:



Autotasker: Self-Service Task Scheduler

Coverage Report





Role-based self-service dashboards for Viewer, Editor, and Admin, each tailored to their responsibilities.



Complete task lifecycle management including creation, editing, scheduling, running, pausing/resuming, and deletion.



Integrated troubleshooting with error-centric views, run history, and one-click retry for faster recovery.



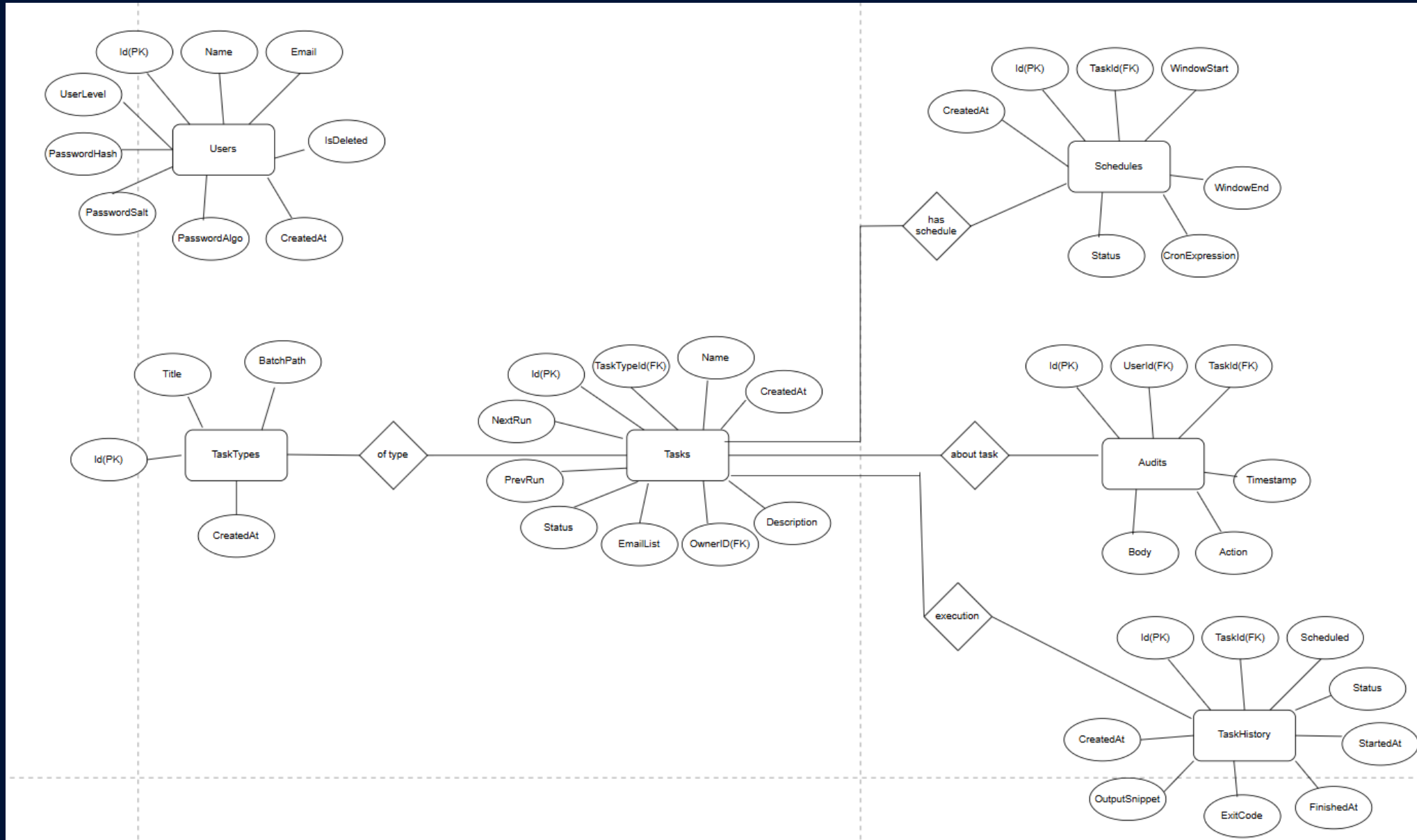
Secure onboarding and strong access governance through password-less user creation and clearly defined permission tiers.



Audit-driven design providing detailed activity logs, improving operational visibility and resolution speed.

Autotasker: Self-Service Task Scheduler

Backend – ER Diagram



Autotasker: Self-Service Task Scheduler

Principles Followed



First Principles Thinking

- Identify clear needs

Agile methodology

- Morning and Evening sprints
- Excel sheet to monitor progress
- Maintaining Documentation



Scientific Thinking

- Identified a problem
- Identified the assumptions
- Framed and tested the hypothesis
- Used clear KPIs to measure success



Assumptions

Assumptions

- Batch Files don't have any parameters
- Client has basic knowledge about tasks, knows how to read logs, and how to troubleshoot basic issues
- Viewers can view all tasks and logs
- Editors can edit all tasks and logs



Use Case Diagram

